



Directory > ... >

Streamlined WAF deployment across
zones and applications

Copy page



Streamlined WAF deployment across zones and applications

Last reviewed: 10 months ago

Introduction

Security perimeters have become less defined compared to the traditional "Castle and Moat" deployments that were popular in the past. Within a fixed perimeter, it was relatively easier to secure multiple applications using a single Web Application Firewall (WAF) deployment inside a datacenter. Today

this approach does not provide enough flexibility as applications and services expand beyond the traditional datacenter. There are several good reasons to configure networks and services in a hybrid approach and to adopt SaaS platforms, so it is valuable to update the WAF approach to cover this scenario.

Cloud-based WAF solutions can control the perimeter sprawl with a flexible deployment model that covers applications and services deployed on-premises, on cloud-based IaaS and PaaS environments, and in hybrid environments.

At the same time, an incorrect implementation of a cloud-based WAF can lead to security policy fragmentation and duplication, causing increased overheads both in maintenance and in monitoring. Aside from the clear economic impact that such inefficiencies bring, the lower efficiency can also degrade the security posture itself. This ultimately can lead to security incidents of varying degrees of severity depending on the scenario.

Who is this document for and what will you learn?

This Design Guide is written for security and network administrators / architects that are looking to implement a flexible, cloud-based WAF security configuration. This configuration can span across multiple applications, domains, and services - all deployed in a hybrid environment.

Cloudflare offers comprehensive Application Security & Performance solutions, which include a highly-configurable, cloud-based Web Application Firewall (WAF).

In this guide, you will learn:

- How to implement the Cloudflare WAF and factor common rules.
- How to easily implement common configurations across multiple applications.
- How to deploy exceptions and specific configurations when needed.

- What are the best practices to follow when deploying the Cloudflare WAF.

Example Scenario

Most Cloudflare customers onboard multiple Cloudflare Zones within a single Account (or Enterprise Organization). Each Cloudflare Zone is usually mapped to a second-level domain (such as `example.com`), and all its subdomains are then handled within that Cloudflare Zone (`web1.example.com`, `web2.example.com` etc.).

In this setup, Cloudflare is a DNS-based reverse proxy. Each Fully Qualified Domain Name (FQDN) is configured in its Cloudflare zone, and it points to a customer origin server. In this respect, Cloudflare does not make particular distinctions on what/where that origin server is located. It could be deployed on-premises, it could be a virtual machine running in the Cloud, or it could be a SaaS service provided by a third-party.

Frequently, multiple FQDNs are pointing to the same, shared web infrastructure. This is reached using an IP address or another FQDN, for example. It is also possible that some FQDNs are pointed at dedicated origin infrastructure or to an external SaaS endpoint.

In many cases, Cloudflare customers end up managing many Cloudflare Zones (such as `example.com`, `example.org`, `myappexample.com` and so on) within a single Cloudflare Account, and many FQDNs within each zone. Frequently, many FQDNs across multiple zones are pointing at a shared web infrastructure behind the scenes.

For example, you could be in the following (or similar) scenario:

- The majority of your web applications run on a newly deployed in-house Content Management System (CMS)
- You also have some legacy web applications that are running on their custom stacks.

- Finally, you may have dedicated infrastructure (managed by a partner) for a few applications.

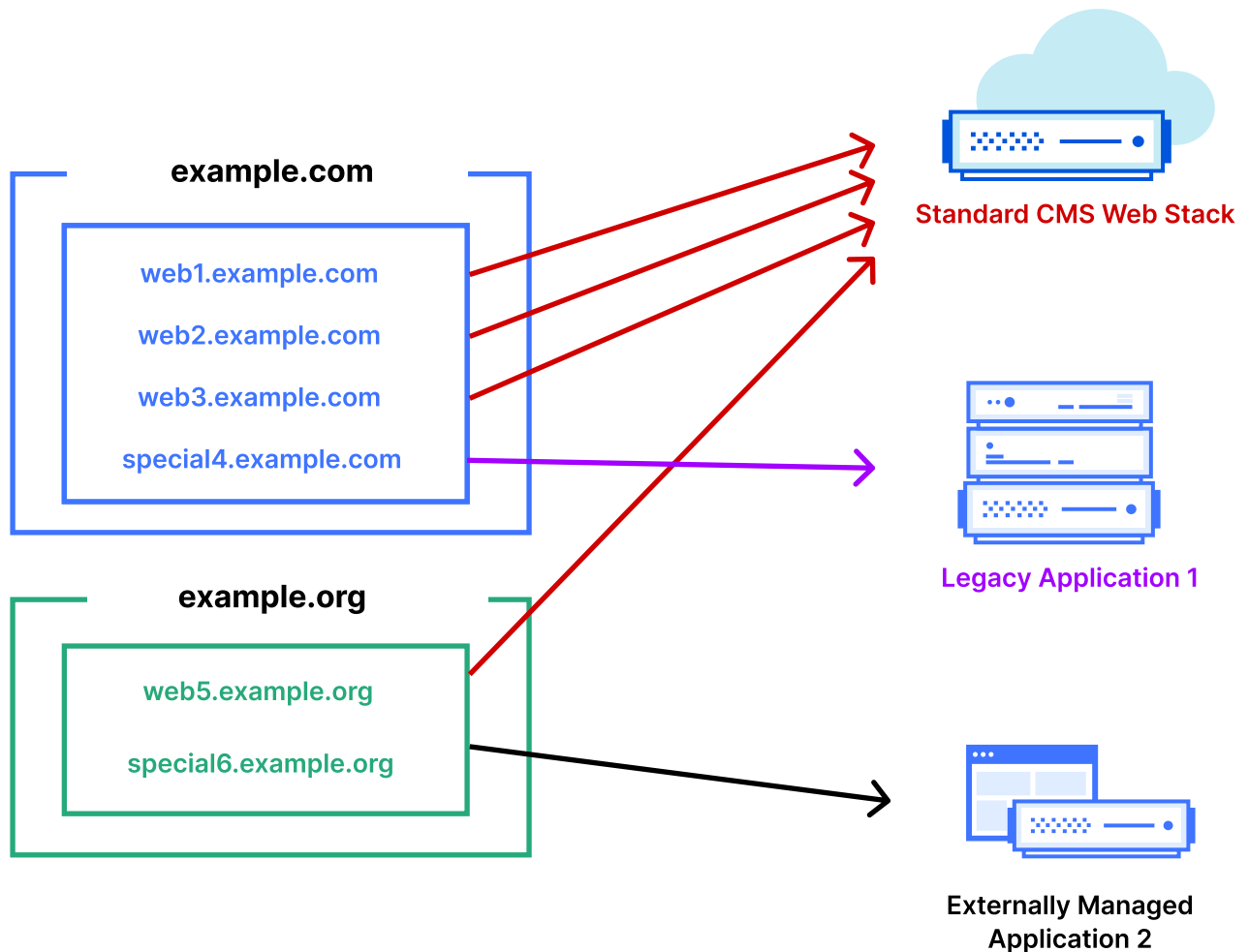


Figure 1: An example scenario with multiple domains, subdomains and web applications.

WAF Requirements

From a WAF setup perspective, this scenario raises interesting requirements:

- To create an easily deployable configuration that implements standard WAF rules configuration in front of most applications.
- To have the ability to fine tune and tweak which rules are deployed in front of the legacy applications, which may be more prone to false positives than the others.
- To include a "catch-all" configuration, ensuring that a Cloudflare default WAF setup is always applied to all web traffic that does not fall in the above scenarios.
- To minimize set up time and ongoing maintenance efforts, as applications are added and removed over time.

In this Design Guide we will review how the Cloudflare WAF operates and what tools are provided to achieve all the above architectural requirements.

Cloudflare Web Application Firewall

The Cloudflare WAF operates at both the zone and the account level. There are different [WAF phases](#) (`http_request_firewall_custom`, `http_ratelimit` and `http_request_firewall_managed`) that map to Custom Rules, Rate Limiting Rules, and Managed Rules. These phases exist both at the account and the zone level. For more information, please [refer to the following documentation](#). It is important to note that the Account rulesets are evaluated before the zone rulesets.

Example Use Case - Implementing the Cloudflare Managed Ruleset

For the purposes of this guide, we will build on the example scenario and WAF Requirements provided above. You have a single Cloudflare Account (or

Enterprise Organization) and two 2nd level domains onboarded on it.

Let's imagine that there are six applications behind six FQDNs across two domains. For these applications, you want to apply a baseline WAF security posture.

However, of these six applications, two will require a more special treatment:

- One is implemented on a legacy application server, prone to false positives.
- Another is implemented by a third party on their own infrastructure.

Let's visualize the scenario below:

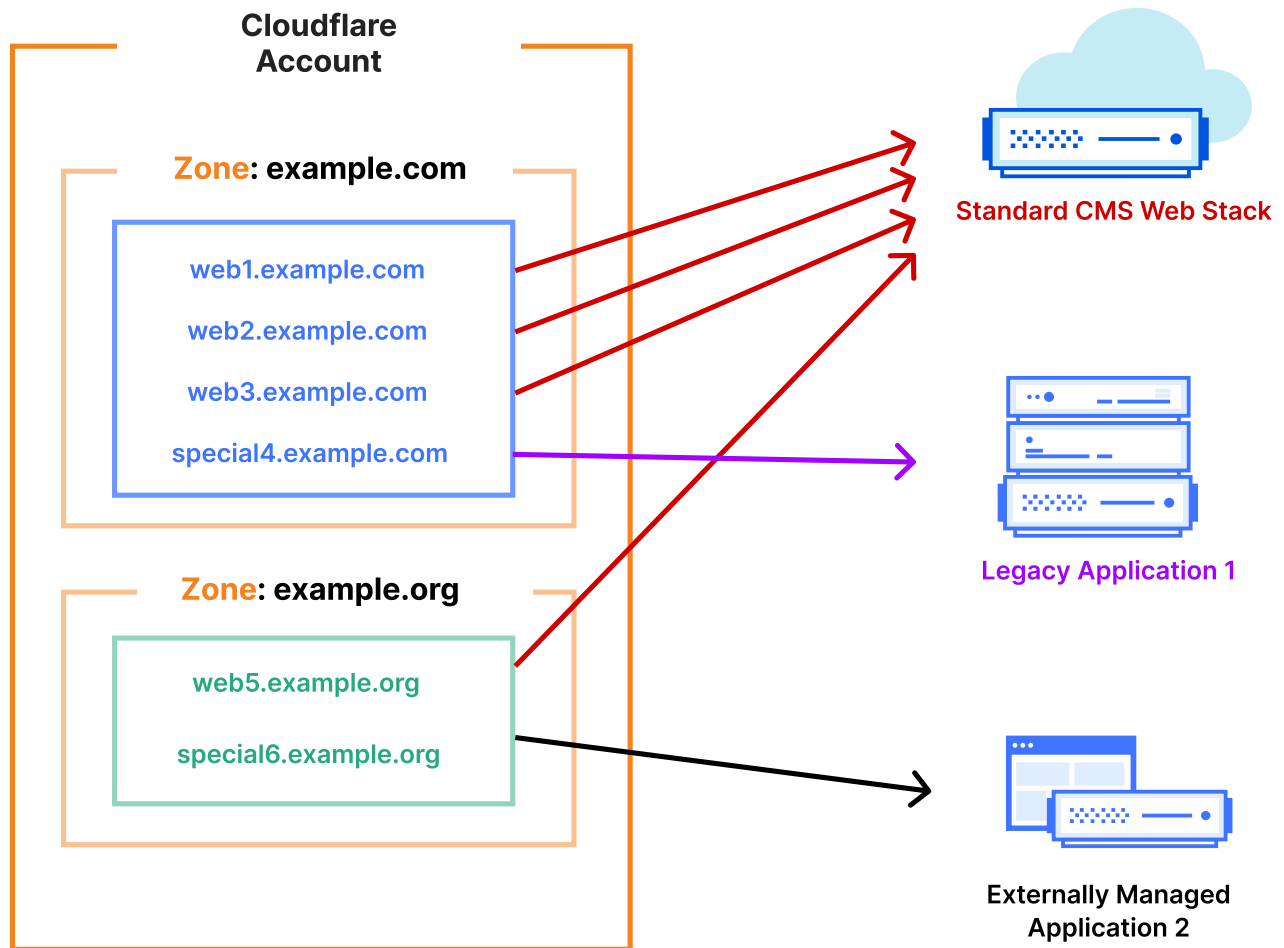


Figure 2: The example scenario now included in a Cloudflare Account with multiple zones.

Using Account Level WAF to minimize configuration overheads

We will use the [Cloudflare Managed Ruleset](#) as an example, keeping in mind that the approach can also

be used for other Cloudflare Managed Rules, Rate Limiting Rules, and Custom Rules.

- For `web1.example.com`, `web2.example.com`, `web3.example.com` and `web5.example.org`: you want to apply the default WAF Managed Ruleset, already tuned by Cloudflare.
- For `special4.example.com`: you want to apply a different subset of the default Managed Ruleset, as you already identified a couple of rules that are causing false positives on the legacy application.
- For `special6.example.org`: you want to apply the Managed Ruleset in logging mode, as this is a newly introduced application from a third party and you need to start evaluating how to protect it.

Then, you can adopt the following approach:

- Deploy one instance of the Cloudflare Managed Ruleset at the Account Level. This implements the

common subset of rules for the four FQDNS requiring it. This is easier to set up and maintain than replicating the same configuration four times at the Zone level.

- For `special4.example.com` and `special6.example.org`, you will deploy two additional instances of the Managed Ruleset, with the specific tweaks required by the applications behind these particular FQDNS.

In practice, using the [Account Level WAF's Managed rulesets](#), you can deploy the three instances of our Managed Ruleset. Each instance will have its own [Custom Filter Expression](#), which will check that the HTTPS requests's hostname belongs to one of the FQDNS in a list:

- For the first list (`web1.example.com`, `web2.example.com`, `web3.example.com` and `web5.example.org`), you will apply the Cloudflare Managed Ruleset in its Default configuration.

- For `special4.example.com`, the same ruleset will be deployed in `Default` mode, but taking care of disabling the specific rules that cause false positives. This can be achieved with the [Rule Overrides](#), using the Dashboard or the APIs. [Real examples are available here](#).
- For `special6.example.org`, you repeat the setup done for the first list, this time modifying the Managed Ruleset instance to operate in `Log` mode instead of `Default`.

Let's visualize the complete configuration in the below diagram:

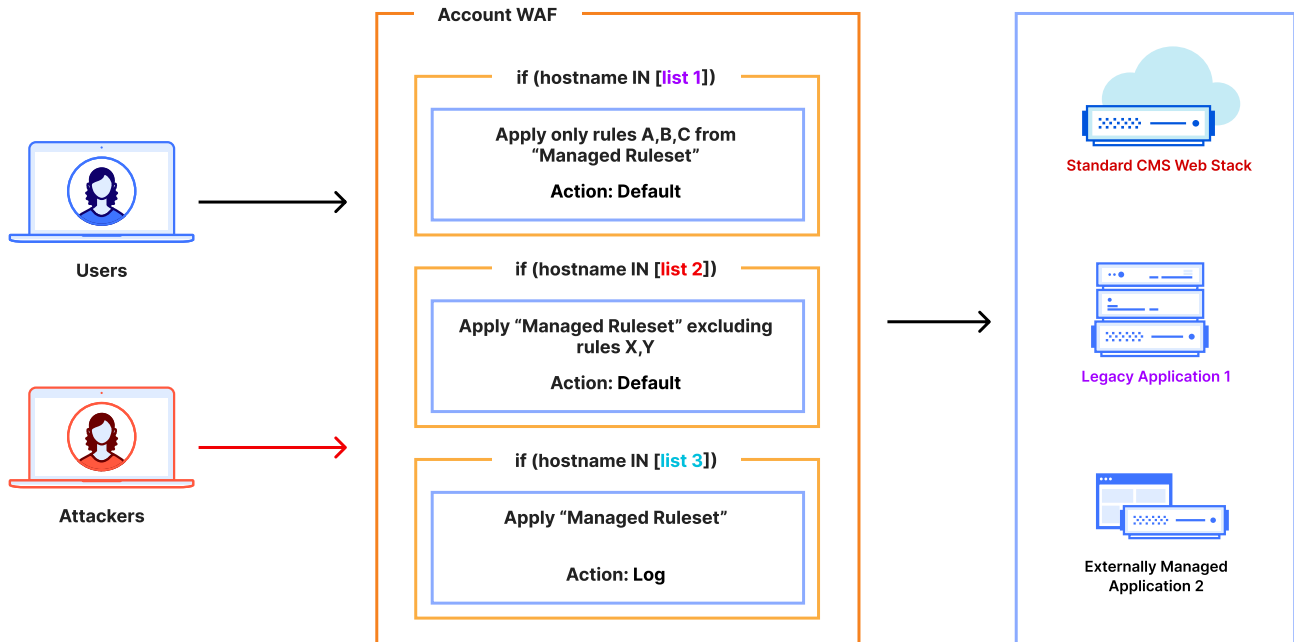


Figure 3: The Account WAF implementation to protect multiple applications across different hostnames with repeatable configurations.

This setup will provide three instances of the Managed Ruleset, calibrated for each application group.

If you have additional applications to be protected in the future, it is sufficient to include the new application FQDN to the filter expression. Generally, most will be added to the standard ruleset instance that is using the recommended Cloudflare configuration. Another common strategy is to add new applications to the Log mode instance, so that it can be monitored and

eventually transitioned to the Default mode ruleset or to a more specific variation if required.

Additional Considerations

False Positives Tuning

The rulesets (and in particular the Managed Ruleset) are already finely tuned by Cloudflare to avoid false positives. They can be deployed for most applications with little to no tweaking required. This means that customers work directly with the default rulesets configurations in most cases, with the possibility to customize only when needed.

If this is your scenario, you can simplify the above setup in the following way by using [Exceptions](#):

- First, you can identify which applications (FQDNs) require a special treatment by deploying the ruleset in Log mode. For example, following testing you find that `special1.example.com`

requires disabling a small set of Managed Rules, and `special2.example.org` disabling a similar, but different set of rules.

- Deploy two managed Exceptions, with a filter matching on the each FQDN, and then skipping those rules from the Managed Ruleset.
- Finally, deploy a Default version of the Managed Ruleset, which will match on everything else, and run the Cloudflare recommended settings of the Managed Ruleset.

This approach can be simpler when there are few exceptions to the norm, and when the initial calibration confirms that the fine tuning already done by Cloudflare to minimize false positives is appropriate in your situation.

Using Lists

Cloudflare provides the ability to create [lists of hostnames](#). In this case, the Filter expression can be changed to reference such list variables.

You can then update the lists directly and re-use them across multiple rulesets. For example, use the same list for the Cloudflare Managed Rules and also for the OWASP Ruleset and Rate Limiting. Your filters [will reference the lists directly](#), meaning a cleaner and maintainable configuration.

When using lists, it is also much easier to adopt a "catch all rule" that runs last in the evaluation order. This could implement, for example, the Default Cloudflare Managed Ruleset when the host in the HTTPS request is not included in any of your lists. This ensures that a default WAF Managed Rules configuration is always applied, in case some of your applications are not added by mistake to the lists.

Using automations

The WAF configuration can be managed [via API calls](#) and [Terraform ↗](#). This is particularly useful when you want to scale the approach to many more zones and

FQDNs, and to avoid repetitive and manual tasks in the Dashboard.

For example, a default Terraform configuration file could be created to define Rulesets and Lists and then maintained and applied as needed without needing to make changes in the Cloudflare Dashboard.

Avoid mixing setup at Account and Zone level

When possible, Cloudflare recommends maintaining the configuration at the Account level, in particular when a Cloudflare Zone will contain multiple DNS records, each requiring custom configuration.

At the Zone level WAF, you can deploy only one instance of each ruleset (Managed Rules, OWASP Rules, etc.), and therefore handling special scenarios can be more complex or not possible at this level.

Custom Rules and Rate Limiting Rules

The approach described above for Managed Rules can be applied also to [Custom Rulesets](#) and [Rate Limiting](#), extending the flexibility to all the WAF security tools at your disposal.

Unless your configuration is specific to a single zone, Cloudflare recommends implementing it at the Account level.

For more information, please refer to the following resources:

- [Create a Rate Limiting Rule at the Account level](#)
- [Create Custom Rulesets at the Account level](#)

Summary

In conclusion, this design guide illustrates how you can implement flexible WAF configurations to cover multiple applications and domains. The described

approach reduces the effort required to deploy, maintain, and update your WAF security configuration.

Was this helpful?



Previous

Next

← **Securing guest wireless networks** **Zero trust architecture for startups** →

Last updated: Oct 9, 2025

Resources

[API](#)

[New to Cloudflare?](#)

[Directory](#)

[Sponsorships](#)

[Open Source](#)

Support

[Help Center](#)

[System Status](#)

[Compliance](#)

[GDPR](#)

Company

Tools

[cloudflare.com](#)[Cloudflare Radar](#)[Our team](#)[Speed Test](#)[Careers](#)[Is BGP Safe Yet?](#)[RPKI Toolkit](#)[Certificate Transparency](#)

Community

[!\[\]\(51b8a815e60dd7bf4906b87368d243e1_img.jpg\) X](#)[!\[\]\(05ebac037cc6375f048d1fb0bccffd53_img.jpg\) Discord](#)[!\[\]\(554da769cf97555ca3a7efb07f40c960_img.jpg\) YouTube](#)[!\[\]\(dcbc5fab1d1aed50d45ce3e946bf9106_img.jpg\) GitHub](#)

© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) • [Cookie Preferences](#)